Écriture d'un entier positif dans une base b >= 2

Système décimal ou base 10

Dans le système décimal, on écrit les nombres à l'aide de nos 10 chiffres bien connus : 0, 1, 2, 3, 4, 5, 6, 7, 8 et 9.

C'est la position du chiffre dans l'écriture d'un nombre qui indique sa valeur. Par exemple, le nombre qui s'écrit 2083 est égal à 2 milliers plus 8 dizaines plus 3 unités. Il n'a pas la même valeur que 8320, même s'il s'écrit avec les mêmes chiffres 0, 2, 8 et 3.

Tout nombre entier naturel peut s'écrire comme combinaison linéaire de puissance de 10. Par exemple, les chiffres du nombre 2083 correspondent à :

chiffre	**2**	**0**	**8**	**3**
i	3	2	1	0
10^i	10^3	10^2	10^1	10^0
combinaison	2×10^3	0×10^2	8×10^1	3×10^0

$$2083 = 2 \times 10^3 + 0 \times 10^2 + 8 \times 10^1 + 3 \times 10^0$$

De manière générale, le nombre n qui s'écrit dans le système décimal avec p chiffres $d_{p-1}d_{p-2}\ldots d_2d_1d_0$ (chaque d_i est un chiffre valant entre 0 et 9) est égal à :

$$n = d_{p-1} \times 10^{p-1} + d_{p-2} \times 10^{p-2} + \ldots + d_2 \times 10^2 + d_1 \times 10^1 + d_0 \times 10^0$$

ou encore avec la formule mathématique d'une somme de 0 à $p-1$: $n = \sum_{i=0}^{p-1} d_i \times 10^i$

Noter qu'on peut écrire 10 nombres allant de 0 à 9 avec 1 seul chiffre, 100 nombres allant de 0 à 99 avec 2 chiffres, 1000 nombres allant de 0 à 999 avec 3 chiffres, ... 10^p nombres allant de 0 à $10^p - 1$ avec p chiffres.

Quand on ajoute un chiffre 0 à droite d'un nombre n, tous les chiffres sont décalés vers la droite, les puissances de 10 correspondantes sont augmentées d'une unité, donc le nombre n est multiplié par 10.

Réciproquement, on peut écrire chacun des chiffres d'un nombre décimal n qui s'écrit dans le système décimal avec p chiffres $d_{p-1}d_{p-2}\ldots d_2d_1d_0$ par un algorithme simple qui consiste à effectuer une succession de divisions entières par 10 :

> L'opérateur de division entière `//` et l'opération modulo `%` utilisés avec des entiers (de type `int`) donnent respectivement le quotient et le reste d'une division euclidienne : si `a` et `b` sont des entiers tels que $a = b \times q + r$, alors `a // b` renvoie q et `a % b` renvoie r.

- Le reste de la division entière de n par 10, `n%10` en Python, renvoie d_0.

- Le quotient de la division entière de n par 10, `n//10` en Python, renvoie $d_{p-1}d_{p-2}\ldots d_2d_1$.

Il suffit alors de répéter l'opération jusqu'à ce que n soit égal à 0, on aura bien obtenu tous les chiffres de l'écriture décimale de n.

Prenons l'exemple du nombre $n = 2083$, le reste de la division entière par 10 est 3 et le quotient 208.

```
>>> 2083%10
3
>>> 2083//10
208
```

```
2083 | 10
   3 | 208
```

On a déjà trouvé le dernier chiffre : 3. Continuons avec 208. Le reste de la division entière de 208 par 10 est 8 et le quotient 20.

```
>>> 208%10
8
>>> 208//10
20
```

```
2083 | 10
   3 | 208 | 10
       8 | 20
```

On obtient le 8. Continuons avec 20. Le reste de la division entière de 20 par 10 est 0 et le quotient 2.

```
>>> 20%10
0
>>> 20//10
2
```

```
2083 | 10
   3 | 208 | 10
       8 | 20 | 10
           0 | 2
```

On obtient le 0. Continuons avec 2. Le reste de la division entière de 2 par 10 est 2 et le quotient 0.

```
>>> 2%10
2
>>> 2//10
0
```

```
2083 | 10
   3 | 208 | 10
       8 | 20 | 10
           0 | 2 | 10
               2 | 0
```

On a obtenu le dernier chiffre 2. Le quotient est 0, inutile de continuer les division, tous les chiffres ont été trouvés.

Noter qu'on a obtenu les chiffres de l'écriture décimale de 2083, mais ⚠️ de gauche à droite, il faut donc les lire de droite à gauche pour retrouver 2083.

```
2083 | 10
   3 | 208 | 10
       8 | 20 | 10
           0 | 2 | 10
               2 | 0
```

On peut traduire cet algorithme en Python, par exemple pour écrire une fonction `etoile` qui renvoie une chaîne de caractère composées de tous les chiffres d'un nombre entier `n` suivis d'une étoile.

```python
def etoile(n):
    n_etoile = ''
    while n > 0:
        n_etoile =  str(n%10) + '*' + n_etoile
        n = n//10
    return n_etoile
```

```
>>> etoile(2083)
'2*0*8*3*'
```

On note que le cas ou `n` est égal à `0`, la fonction n'entre pas dans la boucle `while` et renvoie une chaîne vide. On peut traiter le cas séparément en ajoutant les lignes :

```
def etoile(n):
    if n == 0:
        return '0*'
    n_etoile = ''
    while n > 0:
        ...
```

Exercice corrigé

Un nombre harshad, ou nombre de Niven, est un entier naturel qui est divisible par la somme de ses chiffres dans une base donnée [...]. En base dix, les vingt premiers nombres harshad strictement supérieurs à 10 sont (suite A005349 de l'OEIS) : 12, 18, 20, 21, 24, 27, 30, 36, 40, 42, 45, 48, 50, 54, 60, 63, 70, 72, 80 et 81. Source : https://fr.wikipedia.org/wiki/Nombre_harshad.

Écrire un programme demande un nombre entier et affiche s'il est un nombre de harshad ou pas.

Réponse

Système binaire ou base 2

Cours

En binaire, ou base 2, les seuls chiffres utilisés pour écrire des nombres sont 0 et 1, aussi appelés « bits » pour *binary digits*, autrement dit « chiffres binaires » en français.

8 bits forment un **octet**.

Par exemple on peut écrire 1101, que l'on note aussi 1101_2 pour indiquer qu'il est écrit en binaire. Il convient également de ne pas lire ces nombres comme on lirait des nombres décimaux. Ainsi, 1101_2 ne se dit pas « mille cent un » mais plutôt « un un zéro un ».

Comme dans le système décimal, c'est la position qui indique la valeur de chaque chiffre. Mais en binaire, c'est une combinaison linéaire de puissances de 2. Par exemple, les bits du nombre 1101_2 correspondent à :

bits	**1**	**1**	**0**	**1**
i	3	2	1	0
2^i	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
combinaison	$1 \times 2^3 = 8$	$1 \times 2^2 = 4$	$0 \times 2^1 = 0$	$1 \times 2^0 = 1$

$1101_2 = 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 13_{10}$

Noter le $\ldots_{10}$ pour indiquer que 13 est un nombre en base 10.

Cours

De manière générale, le nombre n qui s'écrit dans le système binaire avec p bits $b_{p-1}b_{p-2}\ldots b_2b_1b_0$ (chaque b_i est un bit valant 0 ou 1) est égal en décimal à :

$n = b_{p-1} \times 2^{p-1} + b_{p-2} \times 2^{p-2} + \ldots + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0$

ou encore avec la formule mathématique d'une somme de 0 à $p-1$: $n = \sum_{i=0}^{p-1} b_i \times 2^i$

On peut écrire les 2 nombres 0 et 1 avec 1 seul bit, 4 nombres allant de 0 à 3 avec 2 bits, 8 nombres allant de 0 à 7 avec 3 bits, ... 2^p nombres allant de 0 à $2^p - 1$ avec p bits.

Exercice corrigé

1. Quel est le plus nombre représentable en binaire sur 16 bits (2 octets) ?
2. Combien de bits doit-on utiliser au minimum pour représenter en base 2 le nombre entier 72 ?

Réponse

Convertir un nombre binaire en décimal

La formule précédente permet de convertir facilement un nombre binaire en décimal. Il suffit de multiplier chaque bit par la puissance de 2 correspondante et de faire la somme des valeurs obtenues.

Énumérons les premiers nombres binaires et quelques autres :

binaire	combinaison	décimal
0	0	0
1	1×2^0	1
10	1×2^1	2
11	$1 \times 2^1 + 1 \times 2^0$	3
100	1×2^2	4
101	$1 \times 2^2 + 1 \times 2^0$	5
110	$1 \times 2^2 + 1 \times 2^1$	6

binaire	combinaison	décimal
111	$1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$	7
1000	1×2^3	8
1 0000	1×2^4	16
10 0000	1×2^5	32
100 0000	1×2^6	64
1000 0000	1×2^7	128
1111 1111	$1 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + \ldots 1 \times 2^0$	255

Exercice corrigé

Calculer la valeur décimale des nombres binaires suivants :

- 11010
- 10101
- 11100110

Réponse

Traduisons cela en Python. Pour plus de simplicité, on peut parcourir le nombre binaire de gauche à droite.

```
def bin_to_dec(n):
    """ str -> int
    Renvoie l'écriture décimale de la chaine de caractère n représentant un nombre binaire
    """
    dec = 0
    for i in range(len(n)):
        dec = dec + int(n[-i-1]) * 2**i
    return dec
```

En Python, les nombres binaires s'écrivent avec le préfixe `0b` et la conversion en décimal est automatique dans la console :

```
>>> 0b1101
13
```

Convertir un nombre décimal en binaire

De la même manière qu'on a précédemment écrit tous les chiffres en base 10 d'un nombre par une succession de divisions entières par 10, on peut écrire un nombre décimal n sous sa forme binaire $b_{p-1}b_{p-2}...b_2b_1b_0$ en effectuant des divisions entières par 2 :

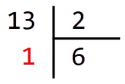
- le reste de la division entière de n par 2, `n%2` en Python, renvoie b_0.
- Le quotient de la division entière de n par 2, `n//2` en Python, renvoie $b_{p-1}b_{p-2}...b_2b_1$.

Il suffit alors de répéter l'opération jusqu'à ce que n soit égal à 0, on aura bien obtenu tous les bits de l'écriture binaire de n.

⚠️ Attention, on obtient les bits de la gauche vers la droite.

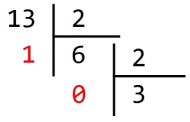
Prenons l'exemple de $n_{10} = 13$, le reste de la division entière par 2 est 1 et le quotient 6.

```
>>> 13%2
1
>>> 13//2
6
```



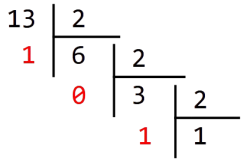
On a déjà trouvé le dernier bit : 1. Continuons avec 6. Le reste de la division entière de 6 par 2 est 0 et le quotient 3.

```
>>> 6%2
0
>>> 6//2
3
```



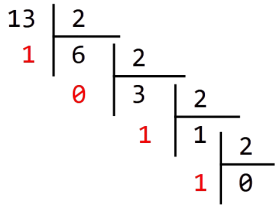
On obtient le bit 0. Continuons avec 3. Le reste de la division entière de 3 par 2 est 1 et le quotient 1.

```
>>> 3%2
1
>>> 3//2
1
```



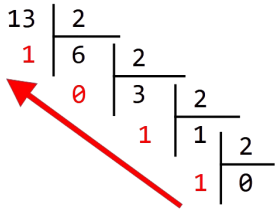
On obtient le bit 1. Continuons avec 1. Le reste de la division entière de 1 par 2 est 1 et le quotient 0.

```
>>> 1%2
1
>>> 1//2
0
```



On a obtenu le dernier bit 2. Le quotient est 0, inutile de continuer les division, tous les bits ont été trouvés.

On a obtenu les bits de l'écriture binaire de 13_{10}, mais ⚠️ de gauche à droite, il faut donc les lire de droite à gauche pour trouver 1101_2.



Exercice corrigé

Écrire les nombres suivants en binaire - 761 - 10101 - 11100110

Réponse

On peut traduire cet algorithme en Python, par exemple pour écrire une fonction `bin_to_dec` qui renvoie l'écriture binaire d'un nombre entier `n` :

```
def dec_to_bin(n):
    """ int -> str
    Renvoie l'écriture binaire de l'entier n
    """
    bin = ''
    while n > 0:
        bin =  str(n%2) + bin
        n = n//2
    return bin


>>> bin(13)
'1101'
```

On note que le cas ou `n` est égal à `0`, la fonction n'entre pas dans la boucle `while` et renvoie une chaîne vide. On peut traiter le cas séparément en ajoutant les lignes :

```
def dec_to_bin(n):
    """ int -> str
    Renvoie l'écriture binaire de l'entier n
    """
    if n == 0:
        return '0'
    bin = ''
    while n > 0:
        bin =  str(n%2) + bin
        n = n//2
    return bin
```

En Python, la fonction `bin` permet d'écrire un nombre décimal en binaire :

```
>>> bin(13)
'0b1101'
```

Opérations

Les techniques des quatre opérations de base (addition, soustraction, multiplication et division) restent exactement les mêmes qu'en notation décimale ; elles sont juste simplifiées de façon drastique parce qu'il n'y a que les deux chiffres 0 et 1. Pour la multiplication par exemple, quelle que soit la base, la multiplication par 10 (c'est-à-dire par la base elle-même)[1] se fait en ajoutant un zéro à droite.

Seules changent d'une part la forme de la suite de chiffres qui exprime le résultat (elle ne compte que des zéros et un), d'autre part la signification de cette suite (10 signifie « deux » et non « dix », 100 signifie « quatre » et non «

cent », etc.).

Addition et soustraction On passe d'un nombre binaire au suivant en ajoutant 1, comme en décimal, sans oublier les retenues et en utilisant la table ordinaire (mais réduite à sa plus simple expression) :

0 + 0 = 0 0 + 1 = 1 1 + 0 = 1 1 + 1 = 0 avec 1 retenue 0 - 0 = 0 0 - 1 = 1 avec 1 retenue 1 - 0 = 1 1 - 1 = 0

Cours

Quand on ajoute un bit 0 à droite d'un nombre n, tous les bits sont décalés vers la droite, les puissances de 2 correspondantes sont augmentées d'une unité, donc le nombre n est multiplié par 2.

Hexadécimal ou base 16

En hexadécimal (base 16), quelle est la valeur de la différence CBD - BAC ? Réponses A AB B TB C FF D 111 Question A.2 Deux entiers positifs ont pour écriture en base 16 : A7 et 84. Quelle est l'écriture en base 16 de leur somme ? Réponses A 1811 B 12B C 13A D A784

Autres bases